As clever as it is, include-and-extend has its own share of problems. For one, each and every module that defines class methods must also define a similar included hook that extends its includer. In a large codebase such as Rails', that hook was replicated over dozens of modules. As a result, people often questioned whether include-and-extend was worth the effort. After all, they observed, you can get the same result by adding one line of code to the includer:

```
class Base
  include Validations
  extend Validations::ClassMethods
  # ...
```

Include-and-extend allows you to skip the extend line and just write the include line. You might argue that removing this line from Base isn't worth the additional complexity in Validations.

However, complexity is not include-and-extend's only shortcoming. The trick also has a deeper issue—one that deserves a close look.

The Problem of Chained Inclusions

Imagine that you include a module that includes another module. You've seen an example of this in *The Validations Modules*: ActiveRecord::Base includes ActiveRecord::Validations, which includes ActiveModel::Validations. What would happen if both modules used the include-and-extend trick? You can find an answer by looking at this minimal example:

part2/chained_inclusions_broken.rb

```
module SecondLevelModule
  def self.included(base)
    base.extend ClassMethods
  end

  def second_level_instance_method; 'ok'; end

  module ClassMethods
    def second_level_class_method; 'ok'; end
  end
end

module FirstLevelModule
  def self.included(base)
    base.extend ClassMethods
  end

  def first_level_instance_method; 'ok'; end
```